

Evaluating Satisfiability Solving with LLMs

ANONYMOUS AUTHOR(S)

Large language models (LLMs) are increasingly used for tasks that implicitly reduce to Boolean satisfiability (SAT), yet their *reasoning* ability on SAT remains unclear. We present a systematic study of LLMs on 2-SAT and 3-SAT, together with two canonical reductions—Vertex Cover and a discrete 3D-packing formulation—designed to probe *representation-invariant* reasoning. Our evaluation begins with the conventional lens (accuracy/precision/recall/F1) and the phase-transition setting. We find that traditional metrics are frequently misleading. Models achieve high scores even though they tend to classify all formulas as satisfiable, fail to reproduce the classical easy–hard–easy signature around the 3-SAT threshold, and degrade sharply as the number of variables N grows.

To address this, we introduce a *paired-formula* protocol (minimally different satisfiable/unsatisfiable instances) and a new measure, *Accurate Differentiation Rate* (ADR), which requires prediction on *both* members of each pair correct. ADR cleanly separates reasoning-oriented models from heuristic ones and correlates with *witness validity* (truth assignments that actually satisfy the formula). Extending beyond CNF, we test cross-representation consistency via standard reductions: (i) Convert CNF to Vertex Cover and (ii) Convert 3-SAT to discrete 3D packing with verifiable placement constraints. Decisions made on CNF and on their graph/packing counterparts agree for most models on $> 80\%$ of instances, revealing stable decision rules across representations. A leading model (e.g., GPT-5) achieves both high invariance and correctness on small N , but still suffers scale-induced degradation.

Taken together, our results support the thesis that SAT is a *conservative* probe for LLM reasoning: performance on SAT predicts transfer to other NP-style reductions, while paired evaluation with ADR provides a faithful, representation-robust assessment beyond conventional metrics.

1 Introduction

Thanks to network architecture and attention mechanism [7, 45], large language models (LLMs) possess strong capabilities to understand contextual relationships among elements within a sequence [10]. Such capabilities enable LLMs to understand not only syntax, but also semantics and context of relevant knowledge, enabling them to excel in solving a wide range of problems.

Reasoning capability is crucial for solving any nontrivial problems. This paper investigates LLMs’ capabilities of solving Boolean satisfiability problems (SAT), both 2SAT and 3SAT. At first sight, it may seem counterintuitive to study this ability, since 2SAT can be solved in polynomial time [2, 44] and modern SAT solvers can handle many large 3SAT instances efficiently in practice [5, 13, 15, 32, 35]. However, for a few reasons, knowing this ability is useful.

First, LLMs are often more convenient to use than standalone SAT solvers. LLMs are easily accessible, whereas SAT solvers require inputs in a formal constraint language (such as CNF/DIMACS). To be solved by SAT solvers, relevant problems need to be first translated into the required format before they can be solved. This involves introducing boolean literals to represent objects in the problem and expressing relations between objects with connectives in SAT formulas. Although translators and modeling frameworks exist for certain domains, using them still requires selecting appropriate abstractions and bounds, validating that the encoding captures the intended semantics, and debugging subtle modeling errors, a process that is similar to building and validating a front-end [1] in a compiler toolchain. As a result, while SAT solvers are widely available and highly efficient once properly instantiated, their effective deployment often demands domain knowledge and engineering effort. In contrast, LLMs lower the barrier by allowing users to describe constraints and obtain candidate solutions directly in natural language.

In fact, LLMs have already been widely used to solve many constraint-heavy problems [40, 52], and a substantial subset of these tasks can be naturally framed as satisfiability or optimization under constraints. For example, scheduling with ternary constraints (such as at most two tasks of X, Y, and Z may be run simultaneously) [18], 3D packing (such as Object X must be next to Y or Z) [29], robotic movement planning (such as avoiding collisions between Objects X, Y, and Z) [26], network configurations [42], and Sudoku puzzles [51] can all be encoded as and are not easier than 3SAT. Simplified versions where constraints are pairwise can be encoded in 2SAT, such as employee shift scheduling and classroom/course timetabling.

Second, LLMs are easier to leverage for developing complex analyses. Some of such analyses include a step that uses SAT solving or is equivalent to SAT solving. For example, semantics-faithful vulnerability detection needs to reason about data-flow analysis [41], alias analysis [39], path feasibility [5], and constant propagation [47]. These analyses are computationally hard due to feasibility conditions involved. Practical tools often discharge such conditions using SAT/SMT backends with potentially iterative refinement loops. Numerous efforts [3, 4, 6, 17, 19, 21, 43, 50] have been made from the static analysis perspective, but the interest in detecting vulnerability using LLMs has gained tremendous momentum, illustrating the shift from solving problems using more rigorous, rule-based analyses to more opportunistic, data-based analyses. This shift largely reflects the simplicity of developing analyses with LLMs. This shift is also happening in other areas, such as program repairing, invariant generation and reasoning [8, 24, 36, 37], program verification [48, 49], and many others [23, 46].

Third, while LLMs and MLs are commonly used in analyses, evaluating performance of such analyses is not easy, due to reasons such as dataset, model parameters, randomness, and class imbalance. For example, in vulnerability detection previous work reported remarkable performance [20, 28, 53, 54] but more recent work revealed that such performance is largely due to noise in existing datasets [14]. For instance, the F1 of StarCode2 [30] decreased from 0.68 for a previous dataset to 0.03 for a more accurate dataset and the performance of GPT-3.5 and GPT-4 is similar to random guessing, even with advanced training techniques [14].

Given the versatility of SAT and that many problems are directly equivalent to or employ SAT as a building block, understanding the ability of SAT solving with LLMs is critical. In particular, we hypothesize that LLM performance on SAT provides a conservative upper bound for performance on a broad class of constraint-based tasks, based on the following rationale.

Consider a constraint-based pipeline that works by (i) deterministically translating a problem instance into one or more SAT formulas, (ii) deciding satisfiability/unsatisfiability of the formula(s), and (iii) deterministically mapping the SAT result back to a solution of the original problem. The pipeline's end-to-end success rate is upper-bounded by the success rate on the required SAT instances. A correct answer is possible only if the SAT subproblems are answered correctly and additional steps such as encoding/decoding and iterative refinement are all free of failure.

Moreover, to generalize robustly in practical settings, ML models must avoid relying on superficial cues and instead reason about the underlying relations and constraints. For many NP-hard/complete tasks, this entails solving an underlying constraint-satisfaction core that can often be encoded as SAT formulas. For example, for vulnerability detection in practical settings, a model has to reason about various data- and control-flows and pointer and alias relations (NP-hard), rather than the presence of certain macros and conditionals (Please see the performance discrepancies discussed a few paragraphs above). As another example, to solve vertex cover, a model has to reason about which vertices cover all edges (NP-hard), rather than relying on statistics such as density or degree distributions.

Thus, when LLMs are used for such tasks in a semantics-faithful manner, their role increasingly resembles the constraint-based pipeline described above. The performance relationship between the

SAT decisions in step (ii) and the pipeline's end-to-end success also applies to LLM-based solutions. In addition to this informal argument, we empirically test the hypothesis in the later sections of the paper.

This paper makes the following contributions.

- (1) We study the problem of SAT solving with LLMs (Sections 3 and 4). Our study involves 12 models, including more recent ones such as GPT-5, and more than 2700 3SAT formulas with varying numbers of variables and difficulties. Our evaluation indicates that current large language models still have very limited abilities of solving satisfiability problems, particularly when the number of variables is more than 25.
- (2) We observe the shortcomings with traditional metrics for measuring machine learning tools, including LLMs, and propose the idea of using paired formulas and a new metric named ADR (Accurate Differentiation Rate) to address the problem (Section 5). Unlike conventional accuracy or F1, which can be inflated by dataset imbalance and superficial cues, ADR directly evaluates whether a model can consistently distinguish paired satisfiable/unsatisfiable instances, thus offering a reasoning-oriented measure. The experiments reveal that ADR precisely captures the LLMs' abilities of satisfiability solving.
- (3) To test the usefulness of ADR, we also investigate an easier logical reasoning problem, namely 2SAT (Section 6). Experiments with more than 700 generated 2SAT formulas show that ADR precisely reflects LLM's abilities of solving this category of logical reasoning problems. Specifically, many reasoning based models, such as GPT 5, DeepSeek-reasoner, and O1, achieve a very high ADR even for problems with more than 50 variables.
- (4) To test the hypothesis that SAT acts as a conservative probe for SAT-reducible tasks, we evaluate the same paired instances with two canonical reductions from SAT to Vertex Cover (Sec. 7) and discrete 3D packing (Sec. 8). We find that LLMs' prediction results for SAT instances and the corresponding counterparts in vertex cover and 3D packing are highly consistent, often greater than 80%. For those the predictions differ, the results for SAT are more likely to be correct than for VC instances and are equally likely to be correct for 3D packing instances. In both tests, ADR remains the most diagnostic signal across representations.

The rest of the paper is organized as follows. We present the necessary background in Section 2, discuss related work in Section 10, discuss threats to validity in Section 11, and conclude in Section 12.

2 Background

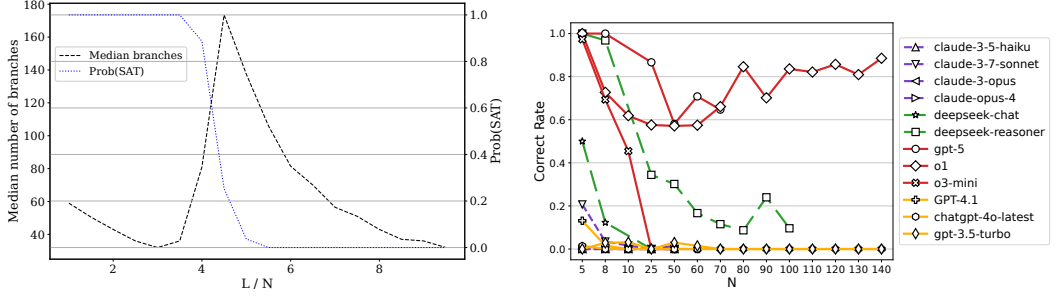
2.1 SAT: Propositional Logic and Conjunctive Normal Form

Propositional syntax. Let $\mathcal{X} = \{x_1, \dots, x_n\}$ be a finite set of propositional variables. A *literal* is either a variable x (a positive literal) or its negation $\neg x$ (a negative literal). A *clause* is a finite disjunction of literals, e.g., $(\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$. A propositional *formula* φ can always be converted into a *conjunctive normal form (CNF)*, which is a finite conjunction of clauses:

$$\varphi = \bigwedge_{i=1}^m C_i \quad \text{with} \quad C_i = \bigvee_{j=1}^{k_i} \ell_{ij}.$$

An *assignment* $\sigma : \mathcal{X} \rightarrow \{\text{false}, \text{true}\}$ assigns false or true to each variable. It extends to literals and clauses in the usual way. A formula φ is *satisfiable (SAT)* if there exists a σ such that assigning variables in φ according to σ makes φ evaluate to true. Otherwise φ is *unsatisfiable (UNSAT)*.¹

¹We write **SAT** (**UNSAT**) to mean a formula is satisfiable (unsatisfiable) and write SAT for all other contexts.



(a) Phase transition for random 3SAT with $N=75$ using a CDCL solver. As the clause-to-variable ratio $\alpha = L/N$ increases, the median number of branching decisions (black, left axis) follows an easy-hard-easy pattern, reaching its maximum near the critical threshold $\alpha_c \approx 4.26$. In contrast, the probability of satisfiability (blue, right axis) remains close to 1 for small α , then undergoes a sharp decline around α_c , and eventually approaches 0 as α grows larger.

(b) Correct rate of low- α UNSAT CNFs' prediction across models as N increases. Compared with o1, whose curve extends to $N = 140$, most other models stop earlier because their performance trends were already evident, making further evaluation at larger N unnecessary.

Fig. 1. (a) Phase transition for random 3SAT with $N=75$ using a CDCL solver and (b) Correct rate of low- α UNSAT CNFs' prediction across models as N increases.

K-CNF and problem classes. If every clause has exactly K literals ($k_i = K$ for all i), the formula is a K -CNF. The decision problems 2SAT and 3SAT ask whether a given 2-CNF or 3-CNF is satisfiable. It is classical traditional that 2SAT is solvable in polynomial time (via implication graphs and strongly connected components) [2], whereas 3SAT is NP-complete [11]. Throughout, we write $N = |X|$ for the number of variables and L for the number of clauses.

2.2 SAT Solvers in Brief

Modern complete SAT solvers descend from the DPLL framework [13] and its Conflict-Driven Clause Learning refinement (CDCL) [32]. At a high level they interleave: (i) *decision* steps that assign a value to an unassigned variable; (ii) *unit propagation* that deterministically forces literals when a clause becomes unit; (iii) *conflict analysis* to derive a *learned clause* (often using the 1-UIP scheme) and perform non-chronological backtracking; and (iv) *restarts* and *branching heuristics* (e.g., VSIDS-style activity scores). Efficient data structures such as *two-watched literals* make propagation nearly linear in the number of clause occurrences. Pre- and in-processing (e.g., subsumption, self-subsuming resolution, blocked-clause elimination, bounded variable elimination) further shrink instances. For 2SAT, linear-time algorithms operate on the implication graph: φ is SAT iff no variable and its negation are in the same strongly connected component [2]. In contrast, 3SAT generally requires the full CDCL arsenal and exhibits the well-known “easy-hard-easy” pattern described next.

2.3 Phase Transition in Random K -SAT

Clause density and criticality. A standard random K -SAT model fixes N and samples L clauses uniformly at random from all K -clauses over N variables. The key control parameter is the *clause density*

$$\alpha = \frac{L}{N}.$$

For $K \geq 3$ there is a sharp satisfiability threshold at a *critical density* $\alpha_c(K)$ in the $n \rightarrow \infty$ limit: for $\alpha \ll \alpha_c(K)$, instances are **SAT** with high probability; for $\alpha \gg \alpha_c(K)$, instances are **UNSAT** with high probability. For $K = 3$, the empirical/analytic consensus places $\alpha_c(3) \approx 4.26$ [27, 34]; for $K = 2$, the threshold is at $\alpha_c(2) = 1$ but the problem remains in P [2]. Around $\alpha_c(K)$, typical algorithmic hardness peaks: solvers require many more decisions/conflicts and runtimes balloon—the canonical *easy-hard-easy* phenomenon [34]. For finite N , one observes a pseudo-threshold $\alpha_c(N)$ (e.g., the location of the 50% satisfiability point or the peak of median solver cost). As N increases, $\alpha_c(N)$ drifts toward the asymptotic threshold $\alpha_c \approx 4.26$, and the transition sharpens; the exact finite-size drift (from above or below) depends on the estimator and the instance generator.

Hardness peak (finite-size view). With finite N , the transition is smoothed by finite-size effects, yet the same qualitative picture holds. Figure 1(a) illustrates this for random 3-SAT with $N=75$ under a modern CDCL solver, namely MiniSat version 2.2[16], which we use to check satisfiability for all formulas in this paper. The probability of satisfiability (blue, right axis) remains close to 1 for small α , then undergoes a sharp decline around the critical threshold $\alpha_c \approx 4.26$, and quickly approaches 0 as α increases further. In parallel, the median number of branching decisions (black, left axis) follows the canonical *easy-hard-easy* pattern, reaching its maximum near α_c . Although finite-size effects smooth the curves, the empirical behavior aligns with the asymptotic threshold phenomenon: the satisfiability transition sharpens and solver hardness concentrates around α_c as N grows larger.

3 INITIAL MISAPPREHENSION, UNDER TRADITIONAL METRICS

Having established in the introduction that SAT solving serves as a critical benchmark for assessing the reasoning capability of large language models (LLMs), we began our investigation by following the conventional evaluation approach used in both classical SAT solving and prior machine learning studies. Specifically, we aimed to observe whether LLMs exhibit the well-known *phase transition phenomenon*, a hallmark of SAT problems, and whether their performance could be adequately captured using traditional classification metrics such as accuracy, precision, recall, and F1.

3.1 Motivation and Research Questions

Our first experiment seeks to answer two questions: **RQ1** Do LLMs demonstrate the same phase transition behavior observed in classical traditional SAT solving? **RQ2** Can traditional evaluation metrics reliably measure the reasoning ability of LLMs on SAT formulas? In classical SAT solving, it is well-established that for a fixed number of variables N , as the clause-to-variable ratio α increases, the satisfiability of formulas undergoes a sharp transition: almost all formulas are satisfiable when α is small, and almost all formulas become unsatisfiable when α is large. Near the critical threshold (approximately $\alpha \approx 4.26$ for 3SAT), the difficulty of solving formulas increases dramatically, requiring significantly more branches (decision points) and conflicts (backtracking steps). Detecting this phenomenon from the results of LLMs is therefore a natural first step for assessing whether they possess genuine reasoning capability in SAT solving.

3.2 Experiment 1 Setup.

To test this, we fixed the number of variables to $N = 75$ and systematically varied α across the range:

$$\alpha \in \{3.5, 4.0, 4.5, 5.0, 5.5\}.$$

We chose these α values to straddle the classical 3-SAT threshold ($\alpha_c \approx 4.26$), thereby covering SAT-dominated, near-critical, and UNSAT-dominated regimes in a single controlled experiment. We fixed $N = 75$ to keep CNFs within prompt-length limits while still exhibiting a clear finite-size

phase-transition signature under a CDCL solver (Fig. 1a). For each α , we generated 320 random 3SAT formulas in CNF form and determined their satisfiability labels using MiniSat. We generated a relatively large number of instances for each α because when α is close to 4.26 it is quite easy for all generated instances to be all **SAT** or **UNSAT**. A large sample instance averts this problem.

```

1 prompt = f"""You are a SAT logic solver.
2     Please use a step-by-step method to solve the following 3-CNF formula.
3
4     Finally, output only the following three items, with no extra explanation:
5     * Whether the formula is SATISFIABLE or UNSATISFIABLE
6     * Number of branches (i.e., decision points)
7     * Number of conflicts (i.e., backtracking steps)
8     If the formula is SATISFIABLE, please give me the value for each variable.
9
10    The formula is:
11    {cnf_text} """

```

Listing 1. LLM prompt for 3-CNF solving

3.3 LLM Prompting Strategy

Each CNF formula was provided to the LLMs through a carefully structured prompt designed to elicit both the satisfiability decision and intermediate reasoning statistics (branches and conflicts). An example prompt is shown in Listing 1. The design intentionally mirrors the behavior of algorithmic solvers, asking the model not only for the final **SAT/UNSAT** classification but also for the reasoning trace, thereby enabling us to compare LLMs’ output with the behavior of algorithmic solvers.

3.4 Observed Outcomes under Traditional Metrics

Compare median branches. As shown in Figure 2(a), median branches of CDCL (black curve), an algorithmic SAT solver, follow an easy–hard–easy pattern with a sharp peak near the critical threshold. In contrast, LLM-reported median branches, as shown in Figure 2(a), do not reproduce the easy–hard–easy pattern. Across LLM models, branch counts remain low or fluctuate idiosyncratically without the expected spike at $\alpha \approx 4.26$. Notably, claude-3-5-haiku is the only model that shows a pronounced increase, but its phase probability curve in Figure 2(b) does not work well. This suggests that the “branches” emitted by LLMs do not reflect solver-like search depth; they are largely artifacts of generated reasoning traces rather than an internal backtracking process.

Compare SAT probability. As shown in Figure 2(b), the black curve (CDCL) exhibits the classical phase transition: the fraction of instances predicted SAT is near 1 for small α , then drops sharply around the threshold (toward 0 as α increases). In contrast, most LLMs produce relatively flat, high curves indicative of a strong **SAT** bias. Most LLM models produce a *flattened or noisy* curve that stays high even where classical CDCL analysis deems almost all instances **UNSAT**. Notably, the gpt-5 curve (red solid line with small circle markers) is the only model that qualitatively follows the CDCL trend—a monotone decrease with α —but its drop is much more gradual and occurs later, failing to reproduce the CDCL-like cliff.

Answer to RQ1. Taken together, these two comparisons indicate that current LLMs do *not* demonstrate the classical traditional phase-transition behavior. While o1 is relatively better—showing some sensitivity to α —it still fails to align with the expected threshold and does not reproduce the canonical shape.

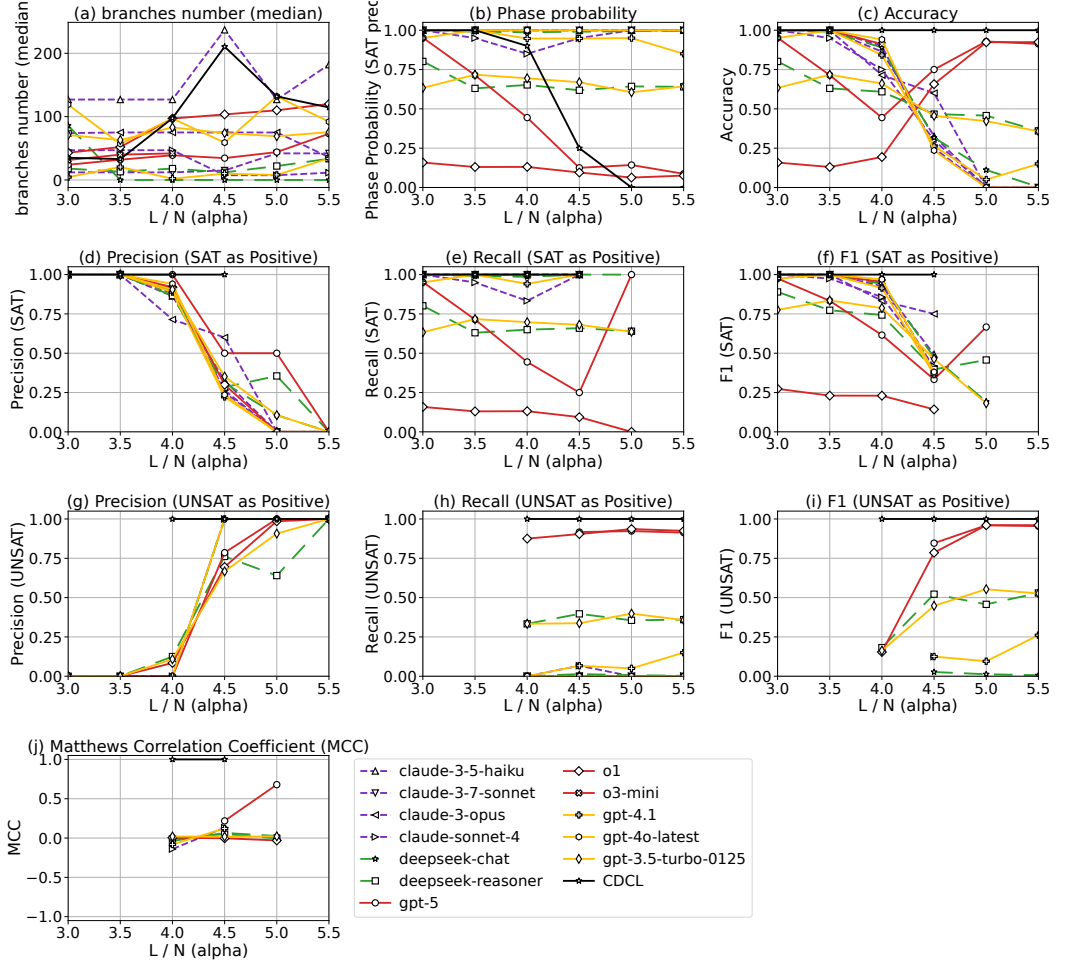


Fig. 2. **Comparison of LLM performance across ten metrics.** 1) Figures in row 2 use SAT as positive and those in row 3 use UNSAT as positive. This makes sure that all data points are presented as α changes from very small (when almost all instances are SAT) to large (when almost all instances are UNSAT). 2) Color/line families: (i) purple dashed = Anthropic Claude (4 variants); (ii) green dashed = DeepSeek (2 variants); (iii) yellow solid = OpenAI non-reasoning models; (iv) red solid = OpenAI reasoning-oriented models. Different marker shapes distinguish individual models within each family. 3) Missing points occur when metrics become undefined, e.g., under highly imbalanced data distributions or when a model consistently predicts SAT, leading to zero denominators in metric formulas.

Compare traditional metrics. After examining Figure 2, panels (c) to (i), we observe:

1) Accuracy (Figure 2 (c)). When $\alpha \leq 3.5$, most LLMs reach high accuracy, many at or near 1.0, with the exception of o1. When $\alpha \geq 5.0$, accuracy for most models drops toward 0; the two clear exceptions are gpt-5 and o1, which remain substantially higher than the rest.

2) Set SAT as the positive class (Figure 2 panels (d) to (f)). For $\alpha \leq 3.5$, almost all LLMs achieve precision = 1 and recall and F1 close to 1, except gpt-5, deepseek-reasoner, and gpt-3.5-turbo-0125. For $\alpha \geq 5.0$, precision of almost all models collapses to 0 or near 0, with gpt-5 and deepseek-reasoner as clear outliers that retain nontrivial precision. In these bins at high α many entries of recall and

F1 are missing, because the bin contains only **UNSAT** instances, which makes these two measures undefined.

3) Set **UNSAT** as the positive class (Figure 2 panels (g) to (i)). For $\alpha \leq 3.5$, precision is 0 for all models, and recall and F1 are undefined (thus blank) because these bins contain only SAT instances. Once $\alpha \geq 4.0$, gpt-5 and o1 obtain relatively strong recall and F1; deepseek-reasoner and gpt-3.5-turbo-0125 show F1 values fluctuating around 0.5; the remaining models stay close to 0.

4) Based on the performance of the models across different metrics, we conclude that except for gpt-5, o1, deepseek-reasoner, and gpt-3.5-turbo-0125, most LLMs exhibit a strong bias toward predicting SAT. Among all models, gpt-5 achieves the best overall performance; the next tier is formed by o1, deepseek-reasoner, and gpt-3.5-turbo-0125. Within this group, o1 tends to predict UNSAT more frequently than the others. These trends are consistent with the class imbalance across α and indicate that high scores in certain bins reflect distributional effects rather than robust logical reasoning.

Compare MCC. Inspecting Figure 2(j), we observe that the Matthews Correlation Coefficient (MCC) [9, 33] is missing at many α values. This is expected: across clause density α , the SAT/UNSAT distribution is highly imbalanced; at small α the bins often contain almost only SAT instances, whereas at large α the bins often contain almost only UNSAT instances. With a limited number of instances per bin, it is common that a bin collapses to a single class. In these degenerate cases the MCC is *undefined*, so we record it as NaN and the corresponding marker is left blank in the plot.

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}.$$

The denominator is a product of four non-negative terms. If any term is zero, MCC is undefined. This occurs whenever a bin has only one class in the ground truth or the model predicts only one class:

- All ground truths are **UNSAT** (no positives): $TP + FN = 0$.
- All ground truths are **SAT** (no negatives): $TN + FP = 0$.
- The model never predicts **SAT** (always **UNSAT**): $TP + FP = 0$.
- The model never predicts **UNSAT** (always **SAT**): $TN + FN = 0$.

Therefore, MCC is only defined (and thus plotted) at α where both the ground truth and the predictions contain *both* classes within the bin.

Answer to RQ2. These observations expose the *initial misapprehension* behind applying traditional classification metrics to LLM-based SAT solving. Accuracy, precision, recall, and F1 are confounded by the α -dependent label imbalance and cannot reliably measure reasoning ability. Even when a model such as o1 numerically outperforms its peers under these metrics, the score improvements are not indicators of solver-like reasoning because the phase transition signature is missing.

In summary, LLM outputs do not replicate the classical phase transition (**RQ1: no**), and conventional metrics fail to provide a trustworthy assessment of reasoning ability in this setting (**RQ2: no**). This motivates our pair-balanced evaluation via *paired formulas* in Section 5.

3.5 Limitations and Lessons Learned

Our findings reveal several fundamental limitations when evaluating LLMs' SAT-solving capability under traditional metrics. These limitations stem from both the inherent properties of SAT distributions and the biases of current LLMs. We summarize two major issues below.

(1) *Imbalanced SAT/UNSAT distributions across α* . As established in classical traditional SAT theory, the satisfiability distribution shifts dramatically with the α clause density [27, 34]: when α is low, nearly all generated formulas are satisfiable; when α is high, nearly all become unsatisfiable. This imbalance means that trivial baselines (e.g., always predicting **SAT** at low α or **UNSAT** at high α) can achieve deceptively high accuracy and recall. Consequently, traditional metrics cannot disentangle genuine reasoning capability from simple exploitation of label imbalance.

(2) *LLMs' strong bias toward predicting SAT*. Across models, we observed a systematic bias in favor of **SAT** predictions, regardless of α . This bias is evident in the “flattened” SAT-probability curves (Figure 2 (b)), which stay high even in regimes where virtually all instances are unsatisfiable. The bias exacerbates the label imbalance issue and further distorts metrics such as precision, recall, and F1. Although o1 departs somewhat from this trivial behavior, its performance still falls far short of solver-like fidelity.

4 Sensitivity to Problem Size and UNSAT Detection

4.1 Research Question

As discussed in Section 3, our initial evaluation revealed two limitations of applying traditional metrics to LLM-based SAT solving. In addition, N was fixed to 75, and thus it was not clear how problem size influences LLM behavior. Without varying N , conclusions about reasoning ability remain incomplete. These motivate the following research question: **RQ3** Can LLMs reliably detect **UNSAT** formulas where **SAT** dominate, and how does scaling the problem size N influence this ability?

4.2 Experiment 2 Setup

To directly address **RQ3**, we deliberately constructed more **UNSAT** formulas at low α , thus isolating LLMs' ability to detect unsatisfiability. At the same time, we varied the number of variables N to examine the effect of scaling problem size. Specifically, we varied N across thirteen settings.

$$N \in \{5, 8, 10, 25, 50, 60, 70, 80, 90, 100, 110, 120, 140\}$$

We varied N over a wide grid up to 140 to expose scale effects while keeping the per- N sample size feasible (70 instances) under API and computation constraints. To explicitly stress **UNSAT** detection in a low- α regime, each generated instance has an $\alpha \in \{3.5, 3.6, 3.7, 3.8, 3.9, 4.0\}$. Also, we discarded all instances whose labels are **SAT** based on a CDCL solver. For each N we generated 70 instances in this manner. We then queried each model with the same prompt as in Experiment 1.

For some models we stop at smaller N once trends are monotone and stable (Fig. 1b) to reserve budget to paired evaluation and cross-representation tests later. While this reaches $N = 140$ for selected models, fully charting the failure boundary at even larger N (or under harder instance distributions) remains constrained by prompt-length limits, timeouts, and API budget; we therefore treat this experiment as a scale stress-test and discuss practical scaling strategies in Sec. 9.

4.3 Results

Figure 1(b) reports the correct prediction rates of low- α **UNSAT** CNFs as N increases. Several trends answer **RQ3**:

1) Negative correlation with problem size. The correct prediction rate declines with larger N , with sharp drops once $N \geq 25$. This shows that scaling remains a fundamental barrier.

2) Reasoning-oriented models perform better. Models such as gpt-5, o1, o3-mini, and deepseek-reasoner consistently outperform general-purpose models. Although their performance also deteriorates, they maintain relatively higher **UNSAT** detection rates.

3) Persistent **SAT** bias in non-reasoning models. Many general-purpose LLMs (e.g., gpt-3.5-turbo, claude-3.5-haiku) fail almost entirely once N increases, confirming their strong bias toward **SAT**.

4.4 Lessons Learned

From these results, we derive three findings regarding **RQ3** :

- 1) Most LLMs cannot reliably detect **UNSAT** formulas when **SAT** dominates the distribution.
- 2) Reasoning-oriented models provide measurable improvements but still degrade significantly with larger N .
- 3) Stress-testing with targeted **UNSAT** examples is essential, as traditional metrics over random distributions cannot reveal these weaknesses.

Thus, the answer to **RQ3** is largely negative: LLMs struggle to detect **UNSAT** formulas at scale, and although reasoning models perform better, they remain limited. This validates the need for distribution-aware and problem-size-sensitive evaluations, and sets the stage for moving beyond aggregate metrics toward paired-formula testing.

5 Refined Understanding with Paired Formulas

5.1 Motivation

As observed in Experiment 1 in Section 3, randomly generated CNFs at the same α exhibit highly imbalanced **SAT** versus **UNSAT** distributions. This imbalance undermines traditional metrics such as accuracy, precision, and recall, since models can exploit label skew rather than possess genuine reasoning to achieve high scores.

These limitations point to a deeper need: an evaluation framework that is both distribution independent and sensitive to minimal structural changes in formulas. To this end, we introduce *paired formulas*—minimally different **SAT/UNSAT** counterparts—and a new evaluation measure, the *Accurate Differentiation Rate (ADR)*, which directly tests whether models can classify both members of a pair correctly.

Essentially, paired formulas eliminate superficial statistical cues and force the model to rely on genuine logical reasoning. Each pair differs by a minimal but logically critical edit (e.g., flipping a literal), so the only way to succeed consistently is to capture the structural shift that collapses or preserves the solution space. ADR operationalizes this by requiring correct classification of both members of each pair, thus serving as a direct proxy for reasoning sensitivity. In this sense, ADR is not merely another accuracy-like metric: it quantifies whether the model truly tracks constraint propagation and logical consistency, rather than exploiting spurious correlations.

Together, paired formulas and ADR raise the following question: **RQ4** Can paired formulas and ADR provide a more reliable evaluation of LLMs' reasoning ability in SAT solving?

5.2 Experiment 3 Setup

For each solver-verified **UNSAT** formula F^{UNSAT} from Sec. 4, we generate a paired **SAT** counterpart F^{SAT} via a *single-edit* procedure, in the following order.

- (1) flip the polarity of one literal in a randomly selected clause such as changing x to $\neg x$ or vice versa;
- (2) replace one literal with another variable while preserving clause width;
- (3) delete one clause.

The edit procedure stopped once an **SAT** instance had been created (verified by MiniSat), which we succeeded for each **UNSAT** instance within these three edits. This procedure kept the variable set fixed and the α values for the paired instances differ to be at most by $\frac{1}{N}$. The full editing script

and edit statistics are given in the supplemental material, which is included in the anonymized package linked from the “Data Availability” Section.

Each **UNSAT** formula F^{UNSAT} was thus paired with a structurally near-identical **SAT** formula F^{SAT} . The generated dataset consists of balanced pairs differing only by minimal but *logically distinct* changes, ensuring that evaluation tested genuine reasoning ability rather than the reliance on superficial heuristics.

5.3 Evaluation Methodology

The same structured prompt was used as in previous experiments. Since the dataset was balanced across pairs, we computed traditional metrics (accuracy, precision, recall, F1) under fair conditions.

In addition, we proposed the *Accurate Differentiation Rate* (ADR), defined as:

$$\text{ADR} = \frac{1}{M} \sum_{i=1}^M \mathbf{1}(\hat{y}(F_i^{\text{SAT}}) = \text{SAT} \wedge \hat{y}(F_i^{\text{UNSAT}}) = \text{UNSAT}).$$

ADR measures whether a model correctly classifies *both* members of a pair. Unlike accuracy or recall, which remain sensitive to prediction bias, ADR directly evaluates whether models can differentiate formulas that are structurally similar but logically distinct. Intuitively, high ADR indicates that a model performs constraint-level reasoning akin to a SAT solver, whereas low ADR suggests reliance on shallow heuristics.

5.4 Comparative Analysis: Discussion of Results

In this section we contrast the proposed *Accurate Differentiation Rate* (ADR) with conventional classification metrics, including accuracy, precision, recall, F1-score, and Matthews correlation coefficient (MCC). Our comparison is based on the paired CNF experiments with varying N s and for each N we generated 70 **SAT/UNSAT** pairs.

5.4.1 Limitations of Traditional Metrics. Figures 3(a)–(h) show that traditional metrics are highly sensitive to class-specific biases and label distributions. For small N (e.g., 5 or 10), all models achieve near-perfect precision, recall, and F1 on **SAT** instances, while their performance on **UNSAT** instances varies widely. This asymmetry artificially inflates overall accuracy, giving the impression of robust reasoning where models are in fact exploiting superficial biases (e.g., over-predicting **SAT**).

Moreover, MCC values (Figure 3(h)) become negative for larger N , reflecting inconsistent predictions across classes. While mathematically sound, such negative correlations are difficult to interpret in terms of reasoning ability. In sum, conventional metrics are easily confounded: high recall on **SAT** can coexist with trivial guessing strategies, while accuracy (Figure 3 (g)) and MCC (Figure 3 (h)) provide contradictory signals under class imbalance.

5.4.2 Advantages of ADR. By design, ADR singles out the joint-correctness signal: it measures the fraction of **SAT/UNSAT** pairs where the model predicts both formulas correctly. As shown in Figure 3(i), ADR aligns with the true reasoning difficulty of the task. For small N , ADR is close to 1, consistent with the models’ ability to handle simple cases. As N grows, ADR decreases smoothly and monotonically, revealing the exact point at which reasoning collapses.

Compared with *Accuracy*, when $N \geq 25$ the accuracies of all models (except GPT-5) cluster around 0.5. This does *not* indicate a reasoning ability of 0.5, because with two classes a random guesser already attains 0.5 accuracy. By contrast, *ADR* faithfully captures the collapse of reasoning for $N \geq 25$, with scores often dropping below 0.2. Moreover, the cross-model ordering—reasoning-oriented models (red/blue curves) lying above non-reasoning ones (yellow)—further shows that

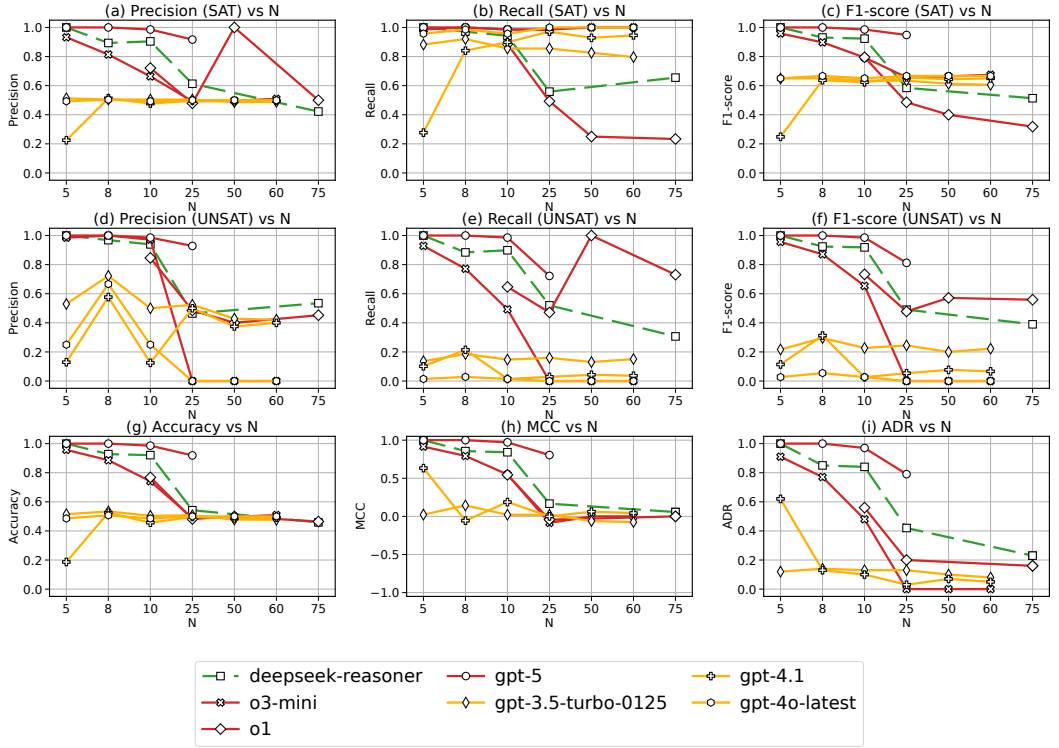


Fig. 3. Performance of different models on paired SAT/UNSAT CNF instances across multiple metrics. **1)** Panels (a)–(h) report traditional classification scores (precision, recall, F1, accuracy, MCC) for SAT and UNSAT separately. **2)** Panel (i) shows the proposed **Accurate Differentiation Rate (ADR)**, which directly measures the fraction of pairs where both SAT and UNSAT predictions are correct. Unlike conventional metrics, ADR declines smoothly with increasing N , revealing the true collapse of reasoning ability and providing a clearer, assessment of model competence. **3)** *Experiment scope:* gpt-5 results stop at $N = 25$ due to frequent timeouts at larger N (which yielded low completion rate), not because ADR ceases to apply. ADR remains well-defined beyond $N = 25$ (e.g., Fig. 3(i) shows meaningful trajectories for deepseek-reasoner and o1 at larger N). In an additional validation run, gpt-5 achieved $\text{ADR} = 0.35$ at $N = 50$ (down from 0.79 at $N = 25$), but this point was omitted from the plot because the $N = 50$ completion rate was only about 20% (most runs timed out). o1 starts from $N \geq 10$ given its validated performance at smaller N and higher API costs; o1 and deepseek-reasoner were run at $N = 75$ (but not $N = 60$), whereas the other models were run at $N = 60$ (but not $N = 75$), primarily due to budget constraints.

reasoning models outperform non-reasoning models. Unlike MCC, ADR never yields negative or ambiguous values, and unlike per-class precision/recall, ADR is invariant to label asymmetries.

5.4.3 Summary. ADR provides a cleaner view of reasoning ability in paired evaluations. Traditional metrics can be inflated by trivial guessing or class imbalance, whereas ADR directly quantifies whether a model can consistently differentiate satisfiable from unsatisfiable counterparts. The experimental evidence confirms that ADR is a more faithful indicator of reasoning competence, especially as problem size increases.

5.5 Lessons Learned

Previous subsections show that paired formulas and ADR provide a more faithful probe of LLM reasoning ability. By controlling for label imbalance and testing sensitivity to minimal structural changes, this methodology reveals whether predictions reflect true logical understanding rather than heuristics.

From a theoretical standpoint, paired formulas function as a “reasoning stress test”: they remove superficial cues and expose whether a model can track the logical consequences of minimal constraint edits. ADR then quantifies this sensitivity, serving as an observable proxy for reasoning strength.

Thus, the answer to **RQ4** is affirmative: while traditional metrics remain confounded by bias, ADR offers a direct window into logical reasoning. Paired formulas and ADR refine our understanding of LLM performance and establish a stronger foundation for future evaluations. Using paired formulas, ADR isolates joint correctness and tracks reasoning difficulty: it is near 1 at small N but drops smoothly with size—non-reasoning models typically fall below 0.2 by $N \geq 25$ while accuracies hover around 0.5 (i.e., random guessing) (Figure 3(i)). Thus, current LLMs do not reliably solve SAT at scale; traditional metrics are confounded by label bias, whereas ADR offers a faithful assessment.

5.6 Mathematical Properties of ADR

We formalize the *Accurate Differentiation Rate (ADR)* in the paired-evaluation setting. Let

$$\{(F_i^{\text{SAT}}, F_i^{\text{UNSAT}})\}_{i=1}^M$$

be M minimally-different pairs and define

$$A_i = \mathbf{1}\{\hat{y}(F_i^{\text{SAT}}) = \text{SAT}\}, \quad B_i = \mathbf{1}\{\hat{y}(F_i^{\text{UNSAT}}) = \text{UNSAT}\}.$$

Per-class recalls and accuracy are

$$r_S = \frac{1}{M} \sum_{i=1}^M A_i, \quad r_U = \frac{1}{M} \sum_{i=1}^M B_i, \quad \text{Acc} = \frac{1}{2}(r_S + r_U).$$

ADR is the *joint-correct* rate over pairs:

$$\text{ADR} = \frac{1}{M} \sum_{i=1}^M A_i B_i.$$

Tight bounds and consequences. ADR satisfies the inequality chain

$$\max(0, r_S + r_U - 1) \leq \text{ADR} \leq \min(r_S, r_U) \leq \text{Acc} = \frac{1}{2}(r_S + r_U).$$

Thus ADR is *stricter* than accuracy/F1: it requires *simultaneous* correctness on both members of a pair and is therefore robust across clause density α [31, 38].

Independence heuristic. Let $A = \{F^{\text{SAT}} \text{ correct}\}$ and $B = \{F^{\text{UNSAT}} \text{ correct}\}$, then

$$\text{ADR} = P(A \cap B) = r_S r_U + \text{Cov}(\mathbf{1}_A, \mathbf{1}_B).$$

, where $\text{Cov}(\mathbf{1}_A, \mathbf{1}_B)$ denotes the covariance of the two indicator variables across pairs. If A and B were independent, $\text{ADR} = r_S r_U$; deviations indicate positive/negative coupling between the two decisions.

Intuitive examples. (1) *Always predict SAT*: $(r_S, r_U) = (1, 0)$ gives $\text{Acc} = 0.5$ but $\text{ADR} = 0$. (2) *One side perfect, one side moderate*: $r_S = 1.0$, $r_U = 0.6$ implies $\text{ADR} = 0.6$ while $\text{Acc} = 0.8$. In both examples, ADR exposes the true *pairwise* limitation that is masked by accuracy/F1.

Summary. ADR cleanly quantifies whether minimal, satisfiability-flipping edits are *jointly* tracked. A detailed derivation of ADR from a mathematical perspective is provided in supplemental material, entitled Mathematical Properties of ADR.

5.7 ADR vs. MCC: a Pair-Level View

Let $M, n_{11}, n_{10}, n_{01}, n_{00}$ denote, respectively, total number of pairs, pairs where both sides are correct, only **SAT**-side is correct, only **UNSAT**-side is correct, or both sides are wrong. Then

$$\text{ADR} = \frac{n_{11}}{M}, \quad r_S = \frac{n_{10} + n_{11}}{M}, \quad r_U = \frac{n_{01} + n_{11}}{M}.$$

On a balanced paired dataset, Matthews correlation coefficient (MCC) takes the closed form below

$$\text{MCC} = \frac{r_S + r_U - 1}{\sqrt{(r_S + 1 - r_U)(r_U + 1 - r_S)}},$$

which becomes *undefined* under class collapse $(r_S, r_U) \in \{(1, 0), (0, 1)\}$ [9, 33].

Let $\beta = \frac{n_{00}}{M}$ (both-wrong frequency) and $\delta = \frac{n_{10} - n_{01}}{M}$ (class asymmetry), then

$$\text{MCC} = \frac{\text{ADR} - \beta}{\sqrt{1 - \delta^2}}.$$

Hence MCC conflates joint correctness (ADR) with both-wrong mass (β) and class asymmetry (δ), whereas ADR isolates the *pairwise joint-correct* component and is always defined. Two models can share the same MCC yet differ markedly in ADR, revealing different discrimination abilities at the pair level.

Summary. ADR should be used as the primary diagnostic of pairwise reasoning and MCC may be used as a secondary, distribution-sensitive correlation summary.

We present further details on the mathematical comparison between ADR and MCC in the supplemental material, titled ADR vs. MCC: A Mathematical Comparison.

6 2-SAT as a Controlled Baseline

6.1 Motivation

As argued in Section 1, SAT is our primary probe of LLM reasoning. While 3SAT is NP-complete and stresses models at modest sizes, 2-SAT lies in P and removes much of the algorithmic hardness. Evaluating 2-SAT next to 3SAT therefore lets us disentangle *reasoning competence* from *problem difficulty* and observe how performance scales with N under a cleaner logical structure.

This motivates the following question: **RQ5** How does evaluating LLMs on 2-SAT, a polynomial-time fragment of SAT, refine our understanding of their reasoning competence compared to 3-SAT, particularly under paired evaluation with ADR?

6.2 Experiment 4 Setup

We construct **70 paired** 2-SAT instances (**SAT/UNSAT**) per setting, with $N \in \{5, 8, 10, 25, 50\}$. Each pair differs by a single satisfiability-changing edit (e.g., literal polarity flip or one-clause tweak), ensuring balanced labels and minimizing superficial cues. We reuse the structured prompts from the 3SAT experiments; models output a categorical decision and, if predicting **SAT**, an assignment to variables.

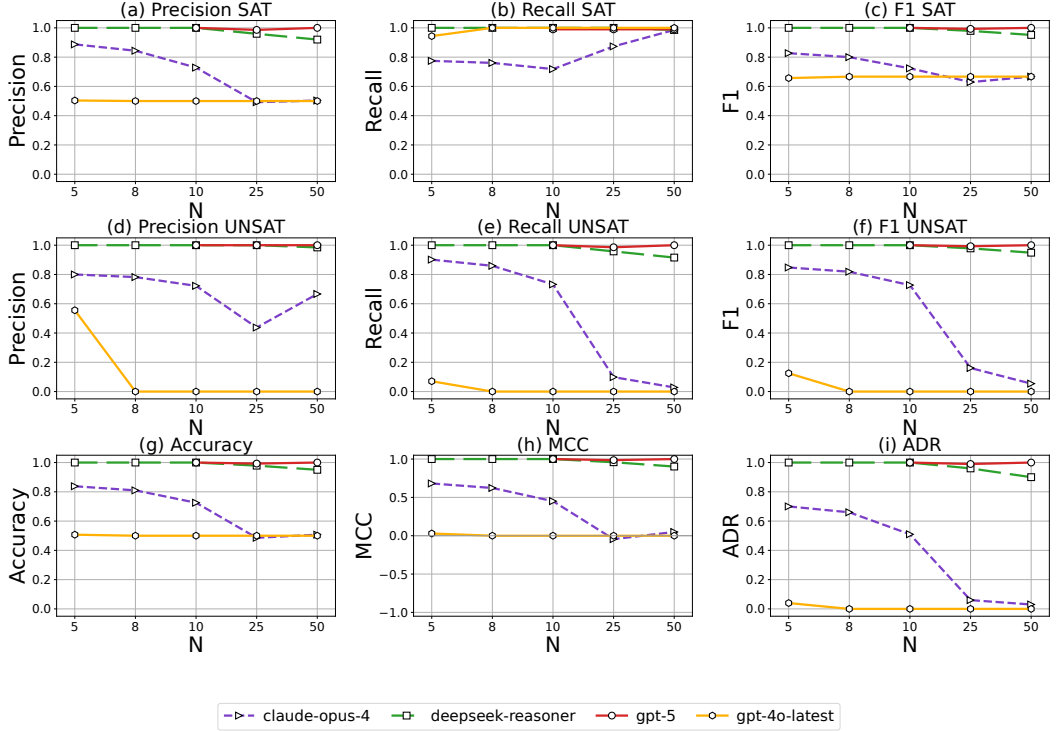


Fig. 4. Performance comparison on 2-SAT instances using Precision, Recall, F1, Accuracy, MCC and ADR metrics. GPT-5 was not evaluated for $N < 10$. Earlier experiments already showed strong performance in this small N regime, so we omitted these settings to conserve API budget.

6.3 Analysis of Results

Figure 4 reveals several important patterns. First, the reduced complexity of 2-SAT lifts conventional metrics at small N : both GPT-5 and deepseek-reasoner achieve near-perfect precision, recall, and F1 on SAT cases (Figure 4 (a–c)), as well as UNSAT precision (Figure 4 (d)) and overall accuracy (Figure 4 (g)). In contrast, claude-opus-4 already shows degradation, confirming that 2-SAT is a simpler fragment but does not erase reasoning requirements. As N increases, gaps widen. GPT-5 sustains strong performance across all metrics, deepseek-reasoner becomes unstable at $N=25$, and claude-opus-4 collapses sharply, with recall and F1 for UNSAT instances (Figure 4 (e–f)) dropping toward zero, reflecting a persistent SAT bias also observed in 3-SAT.

Second, MCC (Figure 4 (h)) corroborates these findings: GPT-5 and deepseek-reasoner retain high correlation with ground-truth labels, while claude-opus-4 hovers near zero, and even negative values emerge as N grows. This alignment across ADR and MCC underscores that pairwise and correlation-based metrics are more diagnostic of genuine reasoning than raw classification scores.

Finally, ADR (Figure 4 (i)) provides a sharper lens than Accuracy or F1. While traditional metrics can appear inflated by one-sided correctness, ADR requires simultaneous correctness on both members of a pair. As a result, ADR clearly separates GPT-5, which remains highly consistent, from deepseek-reasoner, which dips at medium scales, and from claude-opus-4, whose pairwise reasoning fails almost entirely beyond $N=10$.

In summary, 2-SAT validates that lower problem complexity temporarily boosts all models, but scaling still exposes structural weaknesses. ADR and MCC remain the most faithful probes of logical competence, aligning with assignment validity and revealing reasoning robustness that conventional metrics obscure.

6.4 Lessons Learned

Sensitivity to N Across both settings, easier structure yields strong scores at small sizes. For 2SAT, most models reach near-perfect precision/recall/F1 and high accuracy when $N \leq 10$. For 3SAT the same metrics often look healthy at the smallest N but deteriorate much sooner as N grows. The gap widens with scale: 3SAT exposes sharp breakdowns in UNSAT detection and overall stability, whereas 2SAT delays (but does not remove) this collapse.

ADR as the faithful indicator Traditional metrics can appear inflated by one-sided correctness (e.g., overpredicting SAT). In contrast, *Accurate Differentiation Rate (ADR)* declines smoothly with N in both tasks and cleanly separates models that truly track satisfiability-flipping edits from those that rely on superficial heuristics. For 2-SAT, ADR stays high longer (reflecting lower intrinsic difficulty), but it eventually drops. For 3SAT, the ADR decline begins earlier and is steeper, identifying the true onset of reasoning failure that accuracy/F1 may mask.

Bias and MCC behavior A persistent SAT bias is magnified in 3SAT and only partially hidden in 2-SAT. Consequently, MCC may become undefined or volatile in 3-SAT bins with class collapse, and can still be misleading under asymmetric predictions. ADR avoids these pathologies by directly requiring joint correctness on paired instances.

Model ranking and summary Reasoning-oriented models (e.g., GPT-5) maintain top ADR, with 2-SAT showing a higher ceiling and slower decay; others (e.g., deepseek-reasoner) benefit similarly at small N but show instability at intermediate sizes; models with strong SAT bias (e.g., claude-opus-4) collapse under scale, particularly on 3-SAT. Overall, 2SAT serves as a *controlled baseline* that lifts raw scores yet preserves the central message: **paired evaluation with ADR is the most reliable indicator of genuine logical competence, and scaling remains the dominant bottleneck revealed most clearly by 3SAT.**

7 VERTEX COVER: EQUIVALENT OR NOT?

7.1 Motivation and Setup

SAT solving is a foundational probe of reasoning, and many NP problems reduce to it. To test whether LLMs reason in a representation-invariant way, we reduced the paired CNF formulas (Section 5) to *Vertex Cover* instances and asked models to decide feasibility ($|C| \leq k$) and, if YES, to output a cover. Full prompt design, datasets, and protocol are in the supplemental material.

7.2 Key Findings

Most models perform near chance on vectex cover problems, except for GPT-5 (and partly deepseek-reasoner), which achieve strong results on small N but degrade as N increases. We used several ways to compare LLMs' prediction performance for SAT and VC. At the instance level, 75.3% of all SAT instances and their VC counterparts have the exact same prediction results when all LLMs are accumulated. For those that are different, models are more likely to be correct for SAT (73%) than for VC (27%). When measured using ADR, the performance of reasoning models for SAT is much higher than for VC. For example, when $N = 25$, GPT-5's ADR for SAT is 0.79 but that for VC is only around 0.27. Similar performance drop is also observed for deepseek-reasoner.

Overall, this experiment shows that LLM’s performance on SAT is much better than on VC. Also, ADR again provides the clearest signal: it smoothly declines with N and separates reasoning-oriented models from others, while MCC remains noisy or ambiguous.

7.3 Implications

These findings highlight that: (i) SAT invariants transfer to Vertex Cover, supporting its role as a foundational problem; (ii) representation invariance is useful only when paired with genuine competence; (iii) scalability remains the main barrier; and (iv) ADR is the most reliable indicator of reasoning ability across representations. Extended details and analysis appear in the supplemental material.

8 3SAT Transfer to 3D Packing

8.1 Motivation and Setup

To examine *problem-family transfer* beyond CNF and graphs, we reduced the paired 3SAT instances into a discrete *3D packing* formulation whose feasibility exactly mirrors satisfiability. Models were prompted to output both a categorical decision and, when answering YES, a concrete assignment. Full reduction rules, prompt design, and examples are provided in the supplemental material.

8.2 Key Findings

Our experiment results show that most models achieve strong precision/recall/F1 only at small N but quickly degrade with scale; non-reasoning models remain biased toward SAT. Crucially, ADR again proves most diagnostic: reasoning models show high ADR on small instances, followed by a smooth decline toward chance as N grows, while accuracy and MCC often look acceptable despite systematic failures. Moreover, higher ADR correlates with valid packing witnesses, indicating that pairwise discrimination aligns with producing logically correct outputs.

We again compared LLMs’ performance on SAT and on 3D packing at both the instance and the ADR levels. At the instance level, the ratio that LLMs predicted the same results for 3SAT and their corresponding instances in 3D packing was 80.2%. When the prediction results were different, LLMs were equally correct for SAT (53%) and 3D packing (47%). At the ADR level, the performance of reasoners for SAT is again better than that for 3D packing.

8.3 Implications

The packing task preserves the essential difficulty landscape of SAT and confirms that **SAT is a foundational probe of LLM reasoning**: models carry both their strengths and weaknesses across equivalent NP formulations. Traditional metrics can mislead under class imbalance, but ADR reliably tracks reasoning ability and its scale-induced degradation. Detailed construction, metrics, and extended analyses are given in the supplemental material.

9 Discussions

Focus of this paper LLMs have been widely used in solving many problems and reasoning is an important ingredient in problem solving. As emerging models, the reasoning abilities and limitations of LLMs were unclear. Our main focus in this paper includes (i) a systematic empirical study of LLM behaviors on SAT and SAT-derived tasks and (ii) a new evaluation metric (ADR) for testing the faithful reasoning abilities of LLMs when they are applied to pairs of SAT formulas.

Our main insights include the following. First, our empirical study revealed that current LLMs have very limited reasoning ability as problem sizes grow bigger. Second, ADR played an important

role in uncovering these limitations. In particular, under many settings LLM achieved high traditional metric values, but ADR revealed that such achievements were due to **SAT/UNSAT** skew, one-sided prediction bias, or reasoning based on superficial cues, such as clause density. In this sense, ADR served a *foundational evaluation signal*.

Opportunities for LLM–SAT Integration Our results revealed even reasoning LLMs work well only for small instances. Thus, an interesting way of leveraging LLMs is to pursue *solver centered integration*, where SAT solvers provide correctness and state management while LLMs are used only where they add measurable value. SAT solvers can help LLMs by (i) providing deterministic verification of decisions/witnesses and (ii) externalizing heavy symbolic operations (e.g., unit propagation, conflict checking, and state maintenance). This integration supports grounded and stepwise evaluation of LLMs and mitigates their scaling bottlenecks observed in our experiments.

Conversely, LLMs may help SAT solvers as auxiliary policy components for solver control, such as branching heuristics, restart policies, or clause activity scoring. However, our results indicated that LLMs may provide seemingly correct suggestions while they are derived from superficial cues and can be misleading. Thus, before such integration efforts happen, the correctness of LLM suggestions should be carefully investigated. We envision that ADR can be an effective metric for such an investigation.

Applicability of ADR Applying ADR beyond synthetic SAT requires two conditions: (i) a deterministic checker for determining the labels of data instances and (ii) a principled perturbation tool that generates minimally different (or near minimal) pairs whose labels flip. Thus, ADR applies naturally to many SAT derived settings such as graph problems through small edge edits, scheduling and configuration tasks through edits to hard constraints, and program analysis or verification through changes to specifications or source programs.

As a proof of concept beyond random SAT formulas, we applied the same paired evaluation logic to SATLIB “Flat” graph colouring instances [12, 22] (through graph level edits and SAT translation [25]) and evaluated GPT-5. At $N=30$, conventional metrics were moderately high (Precision = 0.78, Recall = 0.76, F1 = 0.77, Accuracy = 0.77), but ADR was only 0.60, again showing that single instance metrics can overstate reasoning abilities.

ADR remains defined even for large instances. Our results already show informative ADR trajectories at larger N for some models (Figure 3). The main bottleneck is not the metric itself but the reliability and cost of obtaining outputs under limitations of prompt length, timeout, and API budget. In general, the problem of applying ADR where LLMs fail to respond reduces to the problem of how to scaling LLMs to deal with large instances. Thus, our idea outlined in leveraging SAT solvers for LLMs also apply here. An alternative approach is to measure the correctness of internal snapshots of an LLM’s reasoning process (if supported by the LLM) rather than the final prediction result only.

Limitations of ADR ADR is designed to measure a model’s abilities to distinguish similar instances with different labels, which often indicates true reasoning abilities. Thus, a main limitation of ADR is the availability of such paired instances. Fortunately, creating paired instances can often be achieved by editing existing instances in small steps. A second limitation of ADR is that the labels of data instances must be accurate. This is, however, a general limitation with all metrics. A third limitation is that ADR is not designed to test end-to-end solver quality. In particular, ADR does not measure runtime, search efficiency, memory usage, or proof complexity. It should also not be viewed as a replacement for solver centered performance metrics and should be viewed as a complement.

10 Related Work

Classical SAT, solvers, and phase transitions. The complexity-theoretic and algorithmic foundations of SAT are well established. 2SAT is solvable in linear time via implication graphs and SCCs [2], whereas 3SAT is NP-complete [11]. Modern complete SAT solvers descend from DPLL [13] and CDCL with clause learning and non-chronological backtracking [32], with subsequent engineering advances such as Chaff [35] and MiniSAT [15]. The *easy-hard-easy* phenomenon and the satisfiability phase transition at fixed clause density $\alpha = L/N$ have been documented for random K -SAT, especially $K=3$ near $\alpha_c \approx 4.26$ [27, 34]. Our study uses these canonical behaviors as a ground truth for assessing whether LLMs exhibit solver-like sensitivity to instance structure.

LLMs for formal reasoning, specifications, and invariants. A growing body of work examines how far LLMs can go in program reasoning tasks. Janßen et al. [24] show that raw LLM outputs are often logically weak in verification settings, limiting usefulness without additional structure. For loop invariants, Chakraborty et al. [8] report that many attempts are typically needed to obtain a correct invariant and introduce a contrastive learning scheme to improve ranking; Pei et al. [36] find that success rates correlate with the availability of small, informative traces. Wu et al. [49] integrate LLMs with automated verification backends and observe difficulties on complex formulas (e.g., nested conditionals and multi-loop cases), while Wen et al. [48] show that unvalidated specification synthesis is frequently invalid and must be coupled with static analysis checks. Overall, these papers point to substantial gaps between fluent generation and *faithful* logical reasoning—an observation consistent with our findings on SAT.

LLMs, datasets, and evaluation pitfalls. Recent evaluations caution that headline metrics can be fragile under dataset noise, class imbalance, and prompt sensitivity. In vulnerability detection, prior work reported strong numbers (e.g., on popular benchmarks), but Ding et al. [14] demonstrate that such gains can collapse on cleaner data: for instance, the F1 of StarCoder 2 drops drastically across datasets [14, 30]. Our results echo this methodological concern in the SAT setting: conventional accuracy/precision/recall/F1 can be inflated by the α -dependent SAT/UNSAT skew, masking reasoning deficits. This motivates our paired-design evaluation and ADR. By construction, each pair always contains one SAT and one UNSAT formula, thereby eliminating class-prior effects.

Reductions and cross-representation reasoning. Many real tasks reduce to SAT or SAT-like cores, including temporal network configuration [42], puzzles such as Sudoku [51], scheduling and planning [18, 26], and packing/placement problems [29]. Classical theory guarantees polynomial reductions among NP problems [11], but little empirical work has tested whether *LLM decisions* remain stable under changes of representation (e.g., CNF $\leftrightarrow$ graphs/packing). Our work fills this gap by measuring *representation invariance* directly: we compare model decisions on CNF with their decisions on standard reductions to Vertex Cover and a discrete 3D-packing formulation, and we relate this to correctness.

Overall, compared to prior LLM-for-verification studies [8, 24, 36, 48, 49], we focus on SAT as a *foundational probe* with a well-understood difficulty landscape [27, 34] and mature solver baselines [13, 15, 32, 35]. Methodologically, we (i) highlight how standard metrics can be misleading under SAT/UNSAT imbalance, (ii) introduce a *paired-formula* protocol with ADR to isolate structure-sensitive discrimination, and (iii) test *cross-representation consistency* via canonical reductions (CNF $\rightarrow$ Vertex Cover, CNF $\rightarrow$ packing). To our knowledge, this combination of phase-transition grounding, pairwise evaluation, and representation-invariance analysis has not been explored for LLM reasoning on SAT.

11 Threats to Validity

Potential threats include errors in reductions, parsing mistakes, or incorrect transformations into 3D-packing instances. We mitigated these by solver-verifying all CNFs, implementing unit-tested reductions, and enforcing strict JSON outputs to decouple reasoning from parsing errors. Traditional metrics (accuracy, precision, recall, F1, MCC) are biased by class imbalance or prediction asymmetry; instead, ADR directly measures pairwise correctness and is always defined. Model self-reported “branches/conflicts” were excluded because they did not align with solver depth.

External and statistical limitations include relatively small sample sizes and budget-constrained repetitions. However, by carefully designing our experiments, such as checking **UNSAT** prediction and using paired formulas, these instances are sufficient to capture large effects but may miss subtle differences. The study focused on random 2SAT/3SAT instances and a limited set of reductions (Vertex Cover, 3D packing), which may not fully represent real-world encodings. However, both problems are practical and are widely used in practice. Also, based on the theory of NP reductions, we believe that the results observed on them do transfer to other equally hard or harder problems.

12 Conclusion

This paper examined whether large language models (LLMs) exhibit solver-like behavior on SAT and closely related formulations. Grounding our evaluation in phase-transition phenomena, we first showed that traditional metrics (accuracy/precision/recall/F1—and even MCC) are misleading under the α -dependent **SAT/UNSAT** imbalance and strong model biases toward **SAT**. LLM self-reported “branches/conflicts” did not track CDCL hardness, and performance degraded sharply with the number of variables N , especially for **UNSAT** detection.

To obtain a view of reasoning, we introduced *paired formulas* and the *Accurate Differentiation Rate* (ADR), which requires simultaneous correctness on minimally different **SAT/UNSAT** instances. ADR cleanly separated reasoning-oriented models from heuristic ones, declined smoothly with N , and aligned with *witness validity* (correct assignments/covers/packings). Beyond CNF, we tested representation invariance via reductions to *Vertex Cover* and a discrete *3D packing* formulation: decisions were largely consistent across representations, and the strongest model achieved both high consistency and truth alignment on small instances—yet all models remained scale-limited.

Current LLMs *do not* solve SAT reliably at scale. Under paired evaluation, ADR is near 1 only for small N and drops monotonically as N grows; non-reasoning models typically fall below ~ 0.2 by $N \geq 25$, and even the best reasoning-oriented models exhibit pronounced degradation. Thus, apparent gains under traditional metrics largely reflect distributional artifacts rather than durable, solver-like reasoning.

Overall, our results support SAT as a *foundational* probe for LLM reasoning and advocate ADR with paired evaluation as a robust complement to conventional metrics. We view three directions as most promising: (i) scale-robust scaffolding that couples LLMs with verified search (e.g., CDCL-style tool use), (ii) training signals that reward pairwise discrimination and verifiable witnesses, and (iii) broader benchmarks that stress representation-invariant reasoning across standard reductions.

Data Availability

All datasets, prompts, supplemental material, and evaluation scripts used in this study are available at: https://anonymous.4open.science/r/ADR_ESSL-C032/README.md For the review phase this link is anonymized; a permanent archival release will be provided upon acceptance.

References

- [1] V Alfred, S Monica, Sethi Ravi, Ullman Jeffrey D, et al. 2007. *Compilers Principles, Techniques*. Pearson.

- [2] Bengt Aspvall, Michael F Plass, and Robert Endre Tarjan. 1979. A linear-time algorithm for testing the truth of certain quantified boolean formulas. *Information processing letters* 8, 3 (1979), 121–123.
- [3] Thomas Ball and Sriram K Rajamani. 2002. The SLAM project: debugging system software via static analysis. In *Proceedings of the 29th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 1–3.
- [4] Al Bessey, Ken Block, Ben Chelf, Andy Chou, Bryan Fulton, Seth Hallem, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson Engler. 2010. A few billion lines of code later: using static analysis to find bugs in the real world. *Commun. ACM* 53, 2 (2010), 66–75.
- [5] Armin Biere, Alessandro Cimatti, Edmund M Clarke, Ofer Strichman, and Yunshan Zhu. 2009. Bounded model checking. Vol. 185. 457–481.
- [6] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O’Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. 2015. Moving fast with software verification. In *NASA Formal Methods Symposium*. Springer, 3–11.
- [7] Fabio Catania, Micol Spitale, and Franca Garzotto. 2023. Conversational agents in therapeutic interventions for neurodevelopmental disorders: a survey. *Comput. Surveys* 55, 10 (2023), 1–34.
- [8] Saikat Chakraborty, Shuvendu Lahiri, Sarah Fakhoury, Akash Lal, Madanlal Musuvathi, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. 2023. Ranking llm-generated loop invariants for program verification. (2023), 9164–9175.
- [9] Davide Chicco and Giuseppe Jurman. 2020. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC genomics* 21, 1 (2020), 6.
- [10] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D Manning. 2019. What does BERT look at? an analysis of BERT’s attention. (2019), 276–286.
- [11] Stephen A Cook. 2023. The complexity of theorem-proving procedures. In *Logic, automata, and computational complexity: The works of Stephen A. Cook*. 143–152.
- [12] Joseph Culberson and Ian Gent. 2001. Frozen development in graph coloring. *Theoretical computer science* 265, 1-2 (2001), 227–264.
- [13] Martin Davis, George Logemann, and Donald Loveland. 1962. A machine program for theorem-proving. *Commun. ACM* 5, 7 (1962), 394–397.
- [14] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2024. Vulnerability detection with code language models: How far are we? *arXiv preprint arXiv:2403.18624* (2024).
- [15] Niklas Eén and Niklas Sörensson. 2003. An extensible SAT-solver. In *International conference on theory and applications of satisfiability testing*. Springer, 502–518.
- [16] Niklas Eén and Niklas Sörensson. 2003. An extensible SAT-solver. In *International conference on theory and applications of satisfiability testing*. Springer, 502–518.
- [17] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. 2001. Bugs as deviant behavior: A general approach to inferring errors in systems code. *ACM SIGOPS Operating Systems Review* 35, 5, 57–72.
- [18] Shimon Even, Alon Itai, and Adi Shamir. 1975. On the complexity of time table and multi-commodity flow problems. (1975), 184–193.
- [19] Tal Garfinkel, Ben Pfaff, Jim Chow, Mendel Rosenblum, and Dan Boneh. 2003. Terra: A virtual machine-based platform for trusted computing. In *Proceedings of the nineteenth ACM symposium on Operating systems principles*. 193–206.
- [20] R Stuart Geiger, Dorothy Howard, and Lilly Irani. 2021. The labor of maintaining and scaling free and open-source software projects. *Proceedings of the ACM on human-computer interaction* 5, CSCW1 (2021), 1–28.
- [21] Thomas A Henzinger, Ranjit Jhala, Rupak Majumdar, and Gregoire Sutre. 2002. Lazy abstraction. In *Proceedings of the 29th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 58–70.
- [22] Holger H Hoos and Thomas Stützle. 2000. SATLIB: An online resource for research on SAT. *Sat 2000* (2000), 283–292.
- [23] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
- [24] Christian Janßen, Cedric Richter, and Heike Wehrheim. 2024. Can ChatGPT support software verification?. In *International Conference on Fundamental Approaches to Software Engineering*. Springer, 266–279.
- [25] Richard M Karp. 2009. Reducibility among combinatorial problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*. Springer, 219–241.
- [26] Henry Kautz and Bart Selman. 1996. Pushing the envelope: Planning, propositional logic, and stochastic search. In *Proceedings of the national conference on artificial intelligence*. 1194–1201.
- [27] Scott Kirkpatrick and Bart Selman. 1994. Critical behavior in the satisfiability of random boolean expressions. *Science* 264, 5163 (1994), 1297–1301.

- [28] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681*.
- [29] Vadim Linetsky. 2004. Spectral expansions for Asian (average price) options. *Operations Research* 52, 6 (2004), 856–867.
- [30] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. Starcoder 2 and the stack v2: The next generation.
- [31] Christopher D Manning. 2008. *Introduction to information retrieval*. Syngress Publishing..
- [32] Joao P Marques-Silva and Karem A Sakallah. 2002. GRASP: A search algorithm for propositional satisfiability. *IEEE Trans. Comput.* 48, 5 (2002), 506–521.
- [33] Brian W Matthews. 1975. Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta (BBA)-Protein Structure* 405, 2 (1975), 442–451.
- [34] David Mitchell, Bart Selman, Hector Levesque, et al. 1992. Hard and easy distributions of SAT problems. In *Aaai*, Vol. 92. 459–465.
- [35] Matthew W Moskewicz, Conor F Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. 2001. Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th annual Design Automation Conference*. 530–535.
- [36] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can large language models reason about program invariants?. In *International Conference on Machine Learning*. PMLR, 27496–27520.
- [37] Muhammad AA Pirzada, Giles Reger, Ahmed Bhayat, and Lucas C Cordeiro. 2024. Llm-generated invariants for bounded model checking without loop unrolling. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1395–1407.
- [38] David MW Powers. 2020. Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. *arXiv preprint arXiv:2010.16061* (2020).
- [39] Ganesan Ramalingam. 1994. The undecidability of aliasing. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 16, 5 (1994), 1467–1471.
- [40] Mubashar Raza, Zarmina Jahangir, Muhammad Bilal Riaz, Muhammad Jasim Saeed, and Muhammad Awais Sattar. 2025. Industrial applications of large language models. *Scientific Reports* 15, 1 (2025), 13755.
- [41] Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise interprocedural dataflow analysis via graph reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 49–61.
- [42] Anirudh Santhiar and Aditya Kanade. 2017. Static deadlock detection for asynchronous C# programs. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 292–305.
- [43] Umesh Shankar, Kunal Talwar, Jeffrey S Foster, and David Wagner. 2001. Detecting format string vulnerabilities with type qualifiers. In *10th USENIX Security Symposium (USENIX Security 01)*.
- [44] Robert Tarjan. 1972. Depth-first search and linear graph algorithms. *SIAM journal on computing* 1, 2 (1972), 146–160.
- [45] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [46] Jiayimei Wang, Tao Ni, Wei-Bin Lee, and Qingchuan Zhao. 2025. A contemporary survey of large language model assisted program analysis. *arXiv preprint arXiv:2502.18474* (2025).
- [47] Mark N Wegman and F Kenneth Zadeck. 1991. Constant propagation with conditional branches. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 13, 2 (1991), 181–210.
- [48] Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. 2024. Enchanting program specification synthesis by large language models using static analysis and program verification. In *International Conference on Computer Aided Verification*. Springer, 302–328.
- [49] Haoze Wu, Clark Barrett, and Nina Narodytska. 2023. Lemur: Integrating large language models in automated program verification. *arXiv preprint arXiv:2310.04870* (2023).
- [50] Yichen Xie and Alex Aiken. 2005. Scalable error detection using boolean satisfiability. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 351–363.
- [51] Takayuki Yato and Takahiro Seta. 2003. Complexity and completeness of finding another solution and its application to puzzles. *IEICE transactions on fundamentals of electronics, communications and computer sciences* 86, 5 (2003), 1052–1060.
- [52] Da Zheng, Lun Du, Junwei Su, Yuchen Tian, Yuqi Zhu, Jintian Zhang, Lanning Wei, Ningyu Zhang, and Huajun Chen. 2025. Knowledge augmented complex problem solving with large language models: A survey.
- [53] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Advances in neural information processing systems* 32.
- [54] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. 2019. VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection. *IEEE Transactions on Dependable and Secure Computing* 18, 5 (2019), 2224–2236.